

NavA11y: A Dynamic Analysis Approach for WCAG 2.4 Focus-Behavior Evaluation

Abu Jafar Saifullah, Tasmia Zerín, Zerina Begum and Kazi Sakib

Institute of Information Technology, University of Dhaka, Dhaka, Bangladesh
{bsse1109, bsse1128, zerin, sakib}@iit.du.ac.bd

Keywords: Web Accessibility, Accessibility Testing, Dynamic Analysis, Keyboard Navigation, Focus Indicators

Abstract: Despite websites becoming integral to daily life, most of them are inaccessible to people with disabilities. While automated techniques exist to detect accessibility violations, prior work primarily relies on static analysis of HTML, CSS, and JavaScript. This analysis fails to determine violations which require live browser interaction. Several violations under the WCAG 2.4 focus-behavior Success Criteria (SCs), such as incorrect tab order, missing or insufficient focus indicators, and element obscuration, arise only during user interaction. To address this gap, we present a dynamic analysis approach targeting all five focus-behavior SCs defined in WCAG 2.4 navigable guidelines. The approach analyzes a webpage at two levels, page level and element level. At the page level, it drives a real browser, simulates keyboard navigation, and presses Tab to record and evaluate the focus order of elements across a webpage. At the element level, style attributes and obscuration ratio are collected for each element. Finally, these are used to check the visibility of a focus indicator and any obscuration by overlapping content. This approach is implemented as a tool called NavA11y. To evaluate this, we constructed a 22-page labeled dataset by extending six focus-related pages from the GDS Accessibility Tool Audit with 16 newly constructed test cases covering all five criteria. NavA11y detected all violations on this dataset without any false positives. To evaluate on real-world cases, we collected 26 production websites. Our approach detected 2,947 violations across those websites, with manual verification on a stratified sample confirming 90% as true positives.

1 Introduction

There are over one billion websites on the world wide web (Internet Live Stats, 2024). Yet, Web Accessibility support is largely overlooked in most websites. An analysis conducted in 2025 revealed that of the top one million home pages, 94.8% had accessibility errors (WebAIM, 2025) with an average of 51 accessibility errors per page. Web Accessibility failures affect approximately 1.3 billion people (16% of the global population) living with a disability (World Health Organization, 2023). This population includes those who rely on keyboard navigation as their primary means of access. A 2024 government audit of 1,151 public-sector websites found that 63.6% failed Web Content Accessibility Guidelines (WCAG) Success Criterion (SC) 2.1.1 (Keyboard) and 76.3% failed SC 2.4.7 (Focus Visible) (Government Digital Service, UK Cabinet Office, 2024a). Although many automated methods exist to detect accessibility violations, none of them cover violations related to keyboard focus behavior.

Keyboard focus behavior has two aspects, both observable only at runtime. The first is the *focus state*: the visible style change a browser applies to an element on receiving focus. It is typically an outline, border change, or background highlight. A static tool can detect whether a focus state is declared in the source. However, it cannot determine whether the browser renders it during navigation, or whether the focused element is obscured by a sticky header (a fixed navigation bar) or an overlay. The second aspect is navigational: the order in which elements receive focus depending on the live Document Object Model (DOM) and active JavaScript event handlers. This order can differ substantially from what the HTML source alone implies. Both aspects leave no trace in the HTML, CSS, or Accessible Rich Internet Applications (ARIA) source. Focus behavior properties are therefore “runtime properties that no screenshot can capture” (He et al., 2025).

WCAG 2.4 (Navigable) defines five success criteria for keyboard focus behavior (W3C, 2023k): SC 2.4.3 (Focus Order), SC 2.4.7 (Focus Visible),

ble), SC 2.4.11 (Focus Not Obscured, Minimum), SC 2.4.12 (Focus Not Obscured, Enhanced), and SC 2.4.13 (Focus Appearance). Evaluating any of them requires a dynamic analysis approach that can drive a real browser, simulate keystrokes, and observe the rendered page state at each focus event. Afterwards, it should be able to accurately analyze the above-mentioned SCs.

In current literature, static tools such as WAVE (WebAIM, 2024a), axe-core (Deque Systems, 2024), Lighthouse (Google LLC, 2024), pally (pally contributors, 2024), and QualWeb (LASIGE, Faculdade de Ciências da Universidade de Lisboa, 2024) inspect HTML, CSS, and ARIA attributes to detect accessibility violations. Large language model (LLM) based tools such as GenA11y (He et al., 2025) go beyond static source inspection by reasoning over rendered screenshots and HTML source. Neither approach can observe how focus moves across a page during keyboard navigation, which is precisely what focus-behavior SCs require. BAGEL (Chiou et al., 2023a) takes a fundamentally different approach. It drives a real browser, simulates Tab key navigation, and analyzes the rendered page layout to detect navigation barriers for SC 2.4.3 and SC 2.4.7. Its SC 2.4.3 analysis centers on whether elements are reachable via keyboard. It does not detect misuse of `tabindex` (an HTML attribute that controls focus order), mismatches between the tab order and DOM order, or cases where CSS moves elements to a different visual position than their DOM order. It also does not address focus-indicator size or contrast (SC 2.4.13), or whether a focused element is obscured by overlapping content (SC 2.4.11, SC 2.4.12).

This paper presents a novel dynamic analysis approach for evaluating all five focus-behavior SCs in WCAG 2.4. This approach is implemented as a tool called **NavA11y**, which drives a real browser via Playwright (Microsoft, 2024) to simulate keyboard navigation and observe the rendered page state at each focus event. The approach consists of two analysis strategies. Page-level analysis (PLA) simulates Tab key traversal across the full page to evaluate focus order (SC 2.4.3). Then it tests this traversal based on DOM order sequence, attribute misuse and incomplete traversal. On the other hand, element-level analysis (ELA) is done to evaluate visibility and obscuration violations. This phase individually focuses each element and captures its rendered focus state and any obscuration by overlapping content (SC 2.4.7, SC 2.4.11, SC 2.4.12, and SC 2.4.13). For evaluating visibility, the captured state is compared with the unfocused state to identify significant change. For

obscuration, a 7×7 grid probe is used to detect any overlapping content on the focused element.

We evaluated NavA11y against a labeled dataset (D_d), consisting of 22 test pages with known ground-truth verdicts, and against a real-world dataset (D_r) of 26 production websites drawn from the Semrush top-site rankings (Semrush, 2024). D_d originally contained 6 focus-related test pages which contained no pass cases, no boundary cases and no obscuration cases. Hence, we extended the original D_d dataset with 16 newly constructed pages to cover all five focus-behavior SCs. NavA11y achieved 100% precision and 100% recall across these test pages. Against the 26 production websites, it detected 2,947 violations across all focus-behavior SCs. A manual evaluation of a stratified sample from six production websites was conducted to validate these findings. Two experienced front-end engineers were asked to verify the accessibility issues detected by NavA11y on these websites. Their independent assessments confirm an average of 90% of reported violations as true positives. Therefore, these results demonstrate that our approach is a viable and effective approach to minimizing the gap in automated accessibility testing.

The paper makes the following two contributions:

- A **dynamic runtime analysis approach** for detecting focus behavior violations, the first approach to cover all five WCAG 2.4 focus-behavior success criteria. We implement this approach as **NavA11y**.
- A **reusable focus behavior dataset** (D_d) containing 22 labeled test pages covering all five focus-behavior success criteria, released for evaluating future tools.

2 Background

The Web Content Accessibility Guidelines (WCAG), published by the World Wide Web Consortium (W3C), define the international standard for web accessibility (W3C, 2023k). WCAG 2.2, the current version, organizes requirements into four principles. These are Perceivable, Operable, Understandable, and Robust, each subdivided into guidelines. Each guideline contains testable statements, called success criteria (SCs), that specify measurable requirements a web page must meet. The criteria span three conformance levels: Level A (the minimum baseline), Level AA (required by most legal frameworks), and Level AAA (the highest standard) (W3C, 2023k). This work focuses exclusively on the Success Criteria related to keyboard focus

behavior within Guideline 2.4 (Navigable) of the WCAG 2.2 standard.

2.1 WCAG 2.4 Navigable Guideline

WCAG 2.2 Guideline 2.4 *Navigable* contains thirteen SCs that specify how users must be able to navigate to and through page content using a keyboard (W3C, 2023k). These 13 criteria fall into two groups: eight concern static properties assessable from the page’s static structure, and five concern keyboard focus behavior observable only at runtime (He et al., 2025).

The static criteria cover page titles, headings, and link labels. The dynamic criteria cover keyboard focus behavior (He et al., 2025).

2.2 Keyboard Navigation and Focus Indicators

Keyboard navigation in browsers is driven by the Tab key, which advances focus to the next interactive element, and Shift+Tab, which moves focus in reverse (W3C, 2023i). The browser tracks the currently focused element and applies a visual focus indicator to it. This indicator is typically an outline, border change, or background highlight, rendered by the browser’s default styles or the page’s `:focus` CSS rule. The examples can be seen in Figure 1.

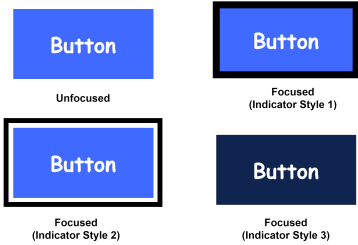


Figure 1: Example of a focused element with different indicator styles

The order in which elements receive focus is determined by the DOM order and any `tabindex` attributes set by the page author (W3C, 2023a). A `tabindex` attribute controls whether an element can receive focus and its position in the Tab sequence. Both the rendered indicator and the focus sequence exist only in the live browser state and are absent from the HTML source.

2.3 Focus-Behavior Success Criteria

The five focus-behavior SCs listed in Table 1 test properties that emerge only after a browser renders the page and responds to keyboard input. The

following paragraphs describe each criterion.

Table 1: Five WCAG 2.4 focus-behavior Success Criteria evaluated by NavA11y.

SC	Full Name	Level
2.4.3	Focus Order	A
2.4.7	Focus Visible	AA
2.4.11	Focus Not Obscured (Minimum)	AA
2.4.12	Focus Not Obscured (Enhanced)	AAA
2.4.13	Focus Appearance	AAA

SC 2.4.3 Focus Order. This criterion requires that a web page can be navigated sequentially using the keyboard and the interactive elements should receive focus in an order that preserves meaning and operability (W3C, 2023i). Browsers determine this order from the sequence in which elements appear in the HTML source (DOM order), but CSS positioning can cause the visual layout to diverge from the source order, creating a mismatch between what the user sees and where keyboard focus moves (W3C, 2023a).

SC 2.4.7 Focus Visible. This criterion requires that when an interactive element receives keyboard focus, a visible focus indicator is displayed (W3C, 2023j). The indicator is typically an outline, border change, or background color shift rendered by the browser’s default styles or the page’s CSS. Indicator appearance depends on how the browser styles the element after focus is triggered. This cannot be determined from the HTML source.

SC 2.4.11 Focus Not Obscured, Minimum (Level AA). This criterion requires that when an element receives keyboard focus, it is not entirely hidden by author-created content such as sticky headers, fixed footers, or cookie banners (W3C, 2023f). At minimum, some part of the focused element must remain visible so the user can locate their position. This can only be determined at runtime.

SC 2.4.12 Focus Not Obscured, Enhanced (Level AAA). This criterion strengthens SC 2.4.11 by requiring that no part of a focused element is hidden by other content (W3C, 2023g). While SC 2.4.11 permits partial obstruction as long as some portion remains visible, SC 2.4.12 requires an element to be fully unobscured. The same runtime constraints apply as in SC 2.4.11.

SC 2.4.13 Focus Appearance (Level AAA). This criterion requires that the focus indicator meets minimum size and contrast requirements, i.e., the indicator must be at least 2 CSS pixels thick and have a contrast ratio of at least 3:1 against the unfocused state and against adjacent colors (W3C, 2023h). Measuring the indicator’s size and color can only be done after the browser has rendered the page and applied focus.

3 Approach

The goal of our approach is to evaluate the five focus-behavior SCs listed in Table 1. We implement it as a tool named **NavA11y**. Given a webpage URL, a browser session is launched and two separate analyses are conducted: Page-Level Analysis and Element-Level Analysis. Page-Level Analysis evaluates SC 2.4.3 (Focus Order). Element-Level Analysis evaluates SC 2.4.7, SC 2.4.11, SC 2.4.12, and SC 2.4.13. The two phases are structurally distinct. SC 2.4.3 requires observing the order in which focus traverses the entire page, which demands a full keyboard traversal. The remaining four SCs require observing each element’s rendered focus state and obscuration individually, which demands direct per-element inspection.

For Page-Level Analysis, Tab key presses are simulated across the page to record the focus path. For Element-Level Analysis, each focusable element, i.e., any element that can receive keyboard focus (links, buttons, and form inputs), is targeted directly to capture its rendered state. The outputs from these phases are passed to the corresponding evaluation checks, as shown in Figure 2. After evaluation, accessibility reports are generated for the webpage. The following subsections describe each phase in detail.

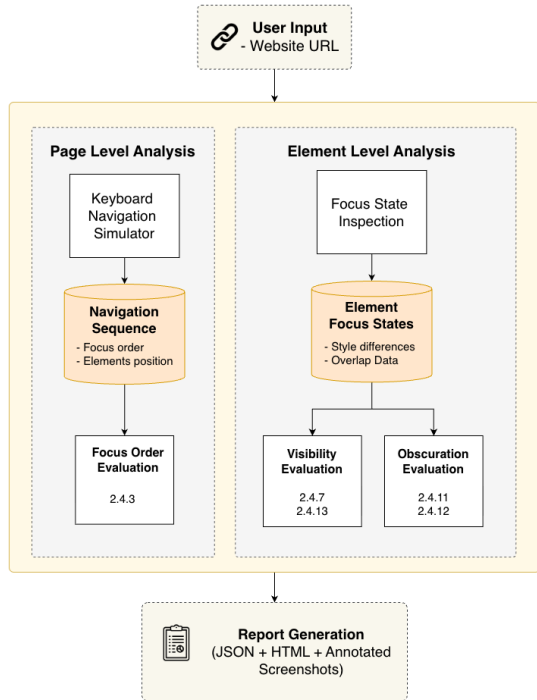


Figure 2: Overview of the NavA11y dynamic analysis architecture.

3.1 Page-Level Analysis

As defined in Section 2, SC 2.4.3 requires that focus moves through the page in an order that preserves meaning and operability (W3C, 2023i). When this order is disrupted, keyboard-only users lose track of their position on the page (W3C, 2023e). Such disruptions are detected by recording three parallel orderings of a page’s focusable elements and testing for divergence between them.

3.1.1 Keyboard Navigation Simulator

A *Navigation Sequence* is recorded, composed of three parallel orderings: DOM order, tab sequence, and visual reading order. The three orderings together cover all known SC 2.4.3 violation patterns (W3C, 2023i; W3C, 2023a). The tab sequence may not follow the HTML source, the visual layout may not match either the HTML source or the tab sequence, or all three may disagree simultaneously. *DOM order* lists every focusable element in the sequence as it appears in the HTML source. It is collected before any Tab key is pressed and serves as the baseline against which the other two orderings are compared.

Tab sequence records the order in which elements receive focus during a simulated keyboard traversal. The traversal stops when focus returns to the start, when a focus trap is detected, or when a limit of 200 key presses is reached. This limit was validated empirically: a pilot study across D_r (the real-world dataset defined in Section 4.1) measured a 95th-percentile DOM focusable element count of 178 elements. The cap of 200 exceeds P95, so pages up to the 95th percentile are covered in full. Only the largest outliers may be truncated. Three consecutive repeats on the same element are flagged as a focus trap. One or two repeats can occur naturally when navigation wraps or a page has few elements, but three consecutive repeats with no forward progress indicate a deadlock.

Visual reading order ranks elements by their screen position, left to right and top to bottom (W3C, 2023a). It captures the spatial layout the user sees, independent of source order or keyboard behavior. These three orderings form the Navigation Sequence and are evaluated next.

3.1.2 Focus Order Evaluation

To evaluate focus order, our approach tests the Navigation Sequence against five independent checks. Multiple checks are needed because SC 2.4.3 violations arise from structurally different causes. Four checks each target a distinct violation pattern: rank

divergence, attribute misuse, CSS repositioning, and incomplete traversal. A fifth check guards against issuing a verdict when the statistical basis is too small to be reliable. Their conditions and thresholds are shown in Table 2.

Order divergence. DOM order is compared with the tab sequence using Kendall τ (Kendall, 1938), a statistical measure of how similar two ranked lists are: 1.0 means identical order; lower values indicate increasing divergence.

To detect significant deviations in focus order, we define a failure threshold of $\tau < 0.90$, which identifies substantial disruptions to the logical navigation sequence. This threshold tolerates minor focus-order variations that can arise at runtime, such as those caused by JavaScript inserting or rearranging a small number of elements after page load (W3C, 2023e). To validate this choice, we conducted a pilot study across all 26 sites in D_r (described in Section 4). All 26 sites completed traversal; 23 had perfect agreement ($\tau = 1.0$). The three sites with non-zero divergence ebay ($\tau = 0.989$), yahoo ($\tau = 0.751$), and qualtrics ($\tau = 0.685$) are each flagged as SC 2.4.3 violations in the production evaluation (Section 4.3). Naturally occurring τ therefore stays at or above 0.989, giving the 0.90 threshold a margin of approximately 0.089 above the observed baseline.

Positive `tabindex`. Any element with a positive `tabindex` is flagged (W3C, 2023b), as it overrides DOM order and disrupts the expected navigation sequence. One exception is skip links (W3C, 2023d). Anchors whose `href` begins with `#` and whose `id` or `class` matches a known pattern (e.g., `skip`, `skip-to-main`) are excluded, since WCAG explicitly recommends their use (W3C, 2023d).

Truncated sequence. If a Tab traversal covers fewer than 60% of a page’s focusable elements, the result is flagged as truncated. This threshold is set by reasoned argument: at 60% coverage, only approximately 36% of all pairwise element relationships are captured (pair counts scale quadratically with element count), leaving the Kendall τ score statistically unreliable.

Small-page guard. Pages with five or fewer focusable elements are routed to manual review (Table 2). The threshold of five is set by reasoned argument: at $N=5$, a single transposed element shifts the Kendall τ score by 0.2 (Kendall, 1938), making every plausible ranking statistically indistinguishable from a violation. For both the truncated-sequence and small-page conditions, no pass or fail verdict is issued. Instead, a warning is recorded in the report to flag the page for manual review.

Visual-order mismatch. CSS can visually move

Table 2: Evaluation conditions with their thresholds.

SC	Condition	Threshold
2.4.3	Kendall τ divergence	$\tau < 0.90$
	Positive <code>tabindex</code>	Any value > 0
	Truncated sequence	$< 60\%$ covered
	Small-page guard	≤ 5 elements
2.4.7	Focus indicator	Any visual change
2.4.11	Obscuration	$\geq 99\%$ covered
2.4.12	Obscuration	Any coverage
2.4.13	Indicator thickness	≥ 2 CSS px
	Indicator contrast	$\geq 3:1$ ratio

an element to a different position on screen without changing where it sits in the HTML source. When this happens, the Tab key follows the HTML order, but the user sees elements in a different order. This is detected by computing Kendall τ between the visual reading order and the tab sequence. A score below the same divergence threshold triggers the flag because a keyboard user cannot predict where focus will land next from what they see on screen (W3C, 2023a).

3.2 Element-Level Analysis

The four element-level SCs check two properties of each focusable element. First, whether a visible focus indicator appears. Second, whether the element is obscured by any overlapping content such as a sticky header or cookie banner. Keyboard users depend on these properties to identify their position on the page (W3C, 2023j; W3C, 2023f). As established in Section 2, both properties are absent from the HTML source and can only be observed at runtime (W3C, 2023c; W3C, 2023f).

3.2.1 Focus State Inspection

For each focusable element, a pre-focus snapshot of computed CSS properties is captured using `getComputedStyle()` (MDN Web Docs, 2026b). The element is then focused using the `page.focus()` API from Playwright (Microsoft, 2024), a browser-automation library, and a post-focus snapshot is captured after a 300 ms delay. Property selection was driven by three WCAG source documents: Technique F78 (a documented failure pattern for CSS-hidden focus indicators) (W3C, 2023c), and the understanding documents for SC 2.4.7 (W3C, 2023j) and SC 2.4.13 (W3C, 2023h). Every CSS property named in those documents in the context of focus indicator appearance or removal was extracted. This yielded six property groups spanning 22 properties: *outline* (style, width, color, offset), *box-shadow*, *border* (per-side width and color), *background* and *text*

Table 3: Grid density comparison across 412 on-screen elements from D_r , measured against the 10×10 reference.

Grid	Probes	SC 2.4.11 agree	SC 2.4.12 agree
5×5	25	100.00%	94.42%
6×6	36	100.00%	98.30%
7×7	49	100.00%	99.03%
8×8	64	100.00%	99.51%
10×10	100	(reference)	

color, text decoration (line, style, color, thickness), and *transform, opacity, filter, visibility, and display*.

The 300 ms delay provides a conservative margin for CSS transitions to settle before the post-focus snapshot is captured (W3C, 2018). A 7×7 grid of 49 evenly spaced points is then placed over the element’s screen area. At each point, the browser’s `elementsFromPoint()` API (MDN Web Docs, 2026a) returns the list of elements at that location, ordered from top to bottom. The topmost element at each point is checked to see whether it is the focused element or overlapping content.

The 7×7 grid size was selected through a pilot study on D_r that tested four grid densities against a 10×10 pixel-level reference across 412 on-screen elements. The study started at 5×5 and increased density until the first grid that reached $\geq 99\%$ agreement on both SC verdicts. As Table 3 shows, 6×6 fell short for SC 2.4.12 at 98.30%. The 7×7 grid was the first to cross that threshold at 99.03%, using 51% fewer probe points than the 10×10 reference (49 vs. 100).

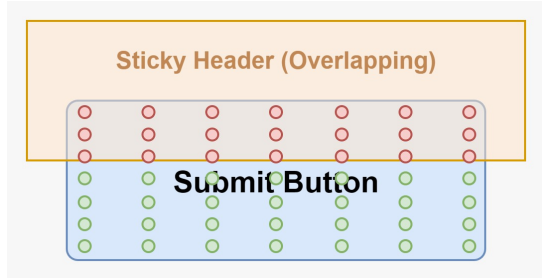


Figure 3: The 7×7 grid probe for obscuration detection. Green points are unobscured. Red points are covered by a sticky header.

The collected style snapshots and obscuration data are evaluated by two independent checks.

3.2.2 Visibility Evaluation

The Visibility Evaluation determines whether a detectable focus indicator appeared when the element received focus. Without any focus indicator, a keyboard user cannot identify which element is currently active (W3C, 2023j). Pre-focus and post-focus style snapshots are compared across the six property

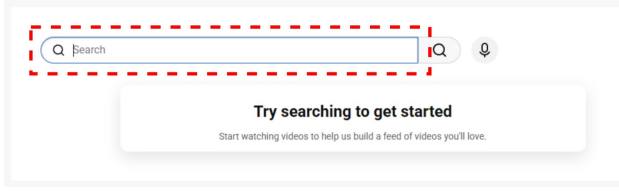
groups identified in Section 3.2: outline, box-shadow, border, background and text color, text decoration, and transform and opacity. We introduce a three-tier confidence classification to distinguish the strength of evidence for each detected change:

- **High:** a change to a dedicated focus-indicator property identified by WCAG Technique F78 (W3C, 2023c), such as `outline` or `border` (e.g., a new outline appears on focus).
- **Medium:** a change to a secondary visual property that can serve as a perceivable indicator but is not a dedicated focus style, such as background color or text decoration (W3C, 2023j).
- **Low:** a change to a property not visually associated with focus indication, such as a CSS transform translation with no accompanying color change.

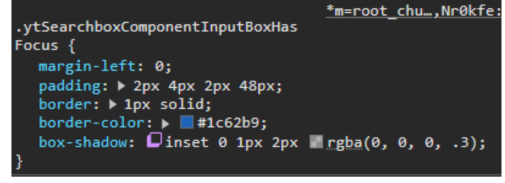
SC 2.4.7 (Level AA) passes if any medium or high-confidence change is found, since the criterion only requires that a focus indicator “is visible” (W3C, 2023j). It fails when only low-confidence changes or no changes are detected. It also fails if a style change removes an existing indicator without adding a new one. This is a failure pattern documented in WCAG Technique F78 (W3C, 2023c). SC 2.4.13 (Level AAA) applies stricter rules from the criterion (W3C, 2023h). The indicator must be at least 2 CSS pixels thick and achieve a contrast ratio of at least 3:1 against the unfocused state (Table 2). The second element-level check evaluates whether overlapping page content obscures the focused element from view, regardless of whether an indicator is present.

3.2.3 Obscuration Evaluation

The Obscuration Evaluation determines whether overlapping page content hides the focused element from the user. Sticky headers, fixed footers, and cookie consent banners commonly overlay page content as the user scrolls, and can cover a focused element so that the user cannot see it or its focus indicator (W3C, 2023f). The 7×7 grid probe is used to measure such obscuration. Each of the 49 points reveals whether the topmost element there belongs to the focused element or to overlapping content. The share of obscured points is the element’s obscuration ratio. SC 2.4.11 (Level AA) fails only when this ratio reaches 99%, which approximates the “entirely hidden” condition in the criterion (W3C, 2023f) while leaving a small margin for pixel rounding. SC 2.4.12 (Level AAA) fails if even one point is obscured, since the criterion requires the element to be fully visible (W3C, 2023g).



(a) Violation when focused on search bar (SC 2.4.7)



(b) CSS styles responsible for violation (SC 2.4.13)

Figure 4: SC 2.4.7 and SC 2.4.13 violations on youtube.com: focused search input exhibits barely visible change in outline (SC 2.4.7), therefore also fails the stricter 2px thickness and 3:1 contrast requirement of SC 2.4.13.

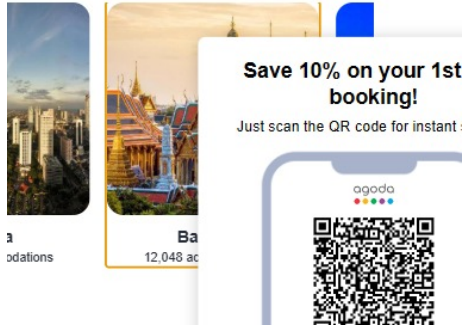


Figure 5: SC 2.4.11 and SC 2.4.12 violations on agoda.com: a focused article card partially hidden beneath a sticky header. SC 2.4.11 fails when the element is entirely hidden; SC 2.4.12 fails as soon as any part is obscured.

3.3 Report Generation

Each finding records the page URL, SC, affected element, and supporting evidence. NavA11y produces three output formats: an HTML report, a machine-readable JSON file, and annotated screenshots.

4 Evaluation

This section reports an empirical evaluation of NavA11y’s detection accuracy, structured around one research question.

RQ: How accurately does NavA11y detect focus-related WCAG 2.4 violations?

It is evaluated on a controlled dataset (D_d) with known verdicts and on production websites (D_r) for real-world performance.

4.1 Experimental Setup

NavA11y¹, which implements the approach described in Section 3, is built in Node.js and uses Playwright v1.58 (Microsoft, 2024). It runs Chromium in

¹Source code available at <https://anonymous.4open.science/r/NavA11y-2C9E>.

headless mode (browser execution without a graphical user interface), making it suitable for automated pipelines (Microsoft, 2024). Three injected scripts capture CSS properties, focus sequence, and obscuration data per page.

D_d — WCAG 2.4 Focus-Behavior Dataset. The UK Government Digital Service (GDS) provides an open-source accessibility dataset (Government Digital Service, UK Cabinet Office, 2024b). Its six focus-related pages cover only a subset of the five SCs and contain no PASS cases, boundary cases, or obscuration scenarios. These were extended with 16 author-constructed pages, giving D_d 22 pages and 32 SC-level assertions spanning all five SCs. Each page was built using the sufficient techniques and failure conditions specified in WCAG (W3C, 2023k), following the GDS methodology (Government Digital Service, UK Cabinet Office, 2024b).

Three categories of test case are represented in the author-constructed pages. PASS cases are included for SC 2.4.3, SC 2.4.7, and SC 2.4.13 to verify that no false positives are raised. SCs 2.4.11 and 2.4.12 have no dedicated PASS pages, since any page without persistent overlays satisfies both criteria by definition (W3C, 2023f; W3C, 2023g). Boundary cases place the page at the exact compliance threshold. For example, one boundary case uses an outline width of exactly 2 CSS pixels, the minimum required by SC 2.4.13 (W3C, 2023h). Obscuration cases cover fixed footers and cookie banners, exercising SCs 2.4.11 and 2.4.12. The dataset can be reused to benchmark future tools.

D_r — Real-World Validation Dataset. A small controlled dataset is insufficient to reveal whether NavA11y produces false positives on complex, dynamic production pages. D_r assesses whether precision is maintained on sites representative of those encountered in practice. The evaluation uses the 30-site Semrush top-website corpus introduced in (He et al., 2025) and sourced from Semrush’s global traffic ranking (Semrush, 2024). These sites represent high-traffic, professionally maintained pages spanning a

range of categories (e.g., banking, e-commerce, fashion). Reusing the same corpus places NavA1ly’s findings in the same benchmark context as (He et al., 2025). Four of the 30 sites were inaccessible to automated evaluation: craigslist.org (geographic IP restriction), doordash (bot-detection pop-up), make-mytrip (HTTP 403 response), and sciencedirect (bot-detection blocking automated navigation), leaving 26 evaluable sites.

Evaluation scope. Each site is evaluated at its home URL using Tab key navigation only, following the same conditions as (He et al., 2025). No login or pre-interaction is applied. Inner pages and authenticated states are not evaluated.

4.2 Results on D_d

NavA1ly correctly classifies all 32 SC-level assertions on D_d : 14 true positives (TP), 18 true negatives (TN), 0 false positives (FP), and 0 false negatives (FN). We report at the assertion level rather than the page level to reveal whether any individual SC is systematically weaker than others. Some pages assert against multiple SCs (e.g., a single obscuration page asserts against both SC 2.4.11 and SC 2.4.12). Each page carries a data-expected attribute. NavA1ly’s output was compared against this attribute to classify each assertion. Table 4 shows the overall classification summary. Table 5 breaks down results per SC. Precision denotes the fraction of reported violations that correspond to genuine failures. Recall denotes the fraction of genuine failures that are detected; F_1 is their harmonic mean.

Table 4: Confusion matrix for NavA1ly on D_d (32 assertions across 22 test pages).

	Predicted Violation	Predicted Pass
Actual Violation	14 (TP)	0 (FN)
Actual Pass	0 (FP)	18 (TN)

Overall: Precision = 100%, Recall = 100%, $F_1 = 1.00$, Accuracy = 100%.

As noted in Section 6, 16 of the 22 pages are author-constructed.

Table 5: Per-SC results on D_d .

SC	TP	TN	FP	FN	Precision	Recall
2.4.3 Focus Order	4	1	0	0	100%	100%
2.4.7 Focus Visible	2	13	0	0	100%	100%
2.4.11 Not Obscured (Min)	2	0	0	0	100%	100%
2.4.12 Not Obscured (Enh)	2	0	0	0	100%	100%
2.4.13 Focus Appearance	4	4	0	0	100%	100%
Total	14	18	0	0	100%	100%

Table 6: Violations detected per website in D_r , grouped by SC.

#	Website	2.4.3	2.4.7	2.4.11	2.4.12	2.4.13	Total
1	adp	0	4	4	13	67	88
2	agoda	1	4	26	62	41	134
3	caliente	1	16	0	2	58	77
4	capitalone	0	8	1	1	73	83
5	cricbuzz	0	0	36	37	185	258
6	discord	0	1	4	5	27	37
7	doubleclick	0	0	0	5	13	18
8	ebay	1	1	0	3	105	110
9	fragrantica	0	3	1	91	365	460
10	genius	1	39	32	32	141	245
11	google	0	2	0	1	20	23
12	live	0	0	0	0	7	7
13	nih.gov	0	0	0	0	3	3
14	openai	0	1	0	2	76	79
15	progressive	0	1	0	0	2	3
16	qualtrics	1	18	6	28	112	165
17	samsung	0	0	0	2	19	21
18	shein	1	5	78	80	5	169
19	stackoverflow	0	0	43	73	114	230
20	steamcommunity	0	3	8	11	190	212
21	usps	0	15	3	4	33	55
22	walmart	0	2	57	57	54	170
23	wikipedia	0	0	0	0	30	30
24	yahoo	1	25	27	33	79	165
25	youtube	0	8	2	7	12	29
26	zerodha	0	0	0	0	76	76
Total		7	156	328	549	1,907	2,947

Table 5 shows that all five SCs achieve 100% precision and 100% recall, yielding $F_1 = 1.00$. Section 4.3 examines whether this precision extends to production websites.

4.3 Results on D_r

NavA1ly detected **2,947 violations** across all 26 D_r sites. Table 6 shows the per-site breakdown. Table 7 summarizes aggregate totals by SC.

Table 7: Aggregate violations on D_r (26 production websites) by SC.

SC	Name	Violations	Sites Affected
2.4.3	Focus Order	7	7/26 (27%)
2.4.7	Focus Visible	156	18/26 (69%)
2.4.11	Not Obscured (Min)	328	15/26 (58%)
2.4.12	Not Obscured (Enh)	549	21/26 (81%)
2.4.13	Focus Appearance	1,907	26/26 (100%)
Total		2,947	26/26 (100%)

SC 2.4.13 (Focus Appearance) accounts for 65% of all violations (1,907 of 2,947), affecting all 26 evaluated sites. Many production websites rely on the browser-default `:focus` outline without customizing it (WebAIM, 2024b). These defaults typically render as a thin 1–2px outline that commonly falls below the required 2CSS pixel thickness or 3:1 contrast ratio (W3C, 2023h). SC 2.4.12 (Focus Not Obscured Enhanced) is the second most prevalent, accounting for 19% of violations (549 across 21 sites), consistent with the persistent overlay patterns described in Section 3.

Violation counts range from 3 (nih.gov) to 460 (fragrantica). Sites with elevated SC 2.4.11 and SC 2.4.12 counts are shein (SC 2.4.11: 78, SC 2.4.12: 80) and walmart (SC 2.4.11: 57, SC 2.4.12: 57). Both websites employed persistent overlays that obstructed numerous focusable elements during the evaluation run. Sites with elevated SC 2.4.13 counts are fragrantica (365) and cricbuzz (185). Both of these websites depended on browser-default focus styles that lacked sufficient indicator contrast or thickness during the evaluation run.

Three categories of edge case affect the D_r results. First, on some sites, JavaScript event handlers trapped or redirected focus before all focusable elements were reached. NavA11y flags these truncated traversals with an advisory notice rather than a pass or fail verdict. Second, dynamic overlays such as coupon dialogs and newsletter pop-ups can appear during Tab traversal and cover focused elements. Because these overlays may appear only on first load or after a timed delay, violation counts for affected sites can vary across runs. The 2,947 violations in Table 6 reflect a single benchmark run. Third, infinite-scroll pages mutate the DOM during traversal, which corrupts style comparisons between pre-focus and post-focus snapshots. This structural limitation is discussed in Section 6.

To assess precision, six sites were selected from D_r using tercile stratification (two per tercile). These were: yahoo and agoda (high), walmart and usps (mid), and zerodha and nih.gov (low). Each reported violation was manually verified against the corresponding WCAG SC definition. For each violation, the raters received the element selector (a CSS path uniquely identifying the element in the DOM), a screenshot of the focused element, and the WCAG SC identifier. They judged whether the reported condition constituted a genuine violation of that criterion.

Prior to labeling, 62 reported violations on yahoo spanning all four element-level SCs (SC 2.4.7, SC 2.4.11, SC 2.4.12, and SC 2.4.13) were identified as infinite-scroll artifacts and excluded. The exclusion criterion was applied systematically. A reported violation was classified as an infinite-scroll artifact if and only if the obscuring element or changed style property was absent from the pre-traversal DOM snapshot captured at page load, indicating it was introduced by a scroll-triggered content load. The remaining 541 reported violations constitute the labeling dataset.

Two experienced front-end engineers independently validated the detected accessibility issues and labeled them as true positives or false positives. The evaluators agreed on 487 out of 541 cases, resulting in an observed agreement of 90%. Cohen’s κ was cal-

culated to assess inter-rater reliability. However, due to the highly imbalanced distribution of labels (i.e., most issues were classified as true positives), κ was unstable and yielded a negative value. Therefore, we report the observed agreement and confusion matrix to better reflect the consistency between evaluators.

Table 8: Inter-rater agreement matrix for manual precision labeling ($N = 541$).

Rater 1 / Rater 2	True Positive	False Positive
True Positive	486	33
False Positive	21	1
$P_o = 0.9002$, $P_e = 0.9016$, $\kappa = -0.0142$		

The negative κ is a known artifact of extreme class imbalance: when one category dominates, P_e approaches P_o and κ becomes unstable (Byrt et al., 1993). To obtain a prevalence-adjusted measure, we compute the Prevalence-Adjusted Bias-Adjusted Kappa (PABAK) (Byrt et al., 1993):

$$\text{PABAK} = 2P_o - 1 = 2 \times 0.9002 - 1 = 0.80$$

A PABAK of 0.80 indicates substantial agreement on the Landis and Koch scale (Landis and Koch, 1977), confirming that the two raters reached consistent verdicts despite the skewed label distribution.

5 Related Work

To our knowledge, no existing tool covers all five WCAG 2.4 focus-behavior SCs. Prior tools address at most two of these SCs (Chiou et al., 2023a; Government Digital Service, UK Cabinet Office, 2024b; Fischer et al., 2025), and none evaluates focus obscuration (SC 2.4.11, SC 2.4.12) or focus appearance (SC 2.4.13).

WebAIM’s WAVE (WebAIM, 2024a) is one of the most widely used accessibility evaluation tools. It uses rule-based checks on a page’s HTML structure to annotate errors and ARIA landmarks. Despite its wide adoption, WAVE automates checks for only 13 of WCAG’s 86 success criteria (WebAIM, 2024a; He et al., 2025; W3C, 2023k); the remainder require manual confirmation.

Axe-core (Deque Systems, 2024) by Deque Systems targets zero false positives by design, ensuring every reported violation is genuine. Every rule fires only when a failure can be confirmed from the static DOM. This design means it systematically under-reports, leaving many violations undetected.

Two widely-used tools wrap axe-core for different deployment contexts. Lighthouse (Google LLC, 2024), built into Chrome DevTools, produces a 0–100

accessibility score and is widely used in CI (continuous integration) pipelines. Pa1ly (pa1ly contributors, 2024) provides a command-line interface for integration into automated test suites. Both inherit axe-core’s rule set and share the same limitation: coverage is bounded by what can be confirmed from the static page structure.

Among static tools, QualWeb (LASIGE, Faculdade de Ciências da Universidade de Lisboa, 2024), developed at the University of Lisbon, offers the broadest formally-specified coverage through the W3C Accessibility Conformance Testing (ACT) framework. It targets SC 2.4.7 (Focus Visible) by inspecting CSS :focus declarations, the rules that control how an element looks when focused. JavaScript can override these styles at runtime. QualWeb cannot observe such overrides, so it misses focus indicators that are applied or removed dynamically.

Benchmarking studies confirm this coverage ceiling. One survey (Fischer et al., 2025) found that the median tool detects only two unique violated success criteria. The UK Government Digital Service audit (Government Digital Service, UK Cabinet Office, 2024b) confirms this finding.

The common limitation across these tools is reliance on static DOM analysis (Fischer et al., 2025). Evaluating focus-behavior criteria requires dynamic analysis, i.e., driving a real browser, simulating keyboard interaction, and observing the rendered page state at each step.

Some prior work has extended the evaluation by triggering interactions during testing. One study (Fernandes et al., 2012) extended the WCAG 2.0 evaluation to Rich Internet Applications (RIAs, pages whose content updates dynamically via JavaScript) by firing JavaScript events and capturing DOM mutations (changes to the live page structure), exposing violations that appear only in post-interaction states. Another (Durgam et al., 2025) detected accessibility anti-patterns called accessibility smells. This is done by recording interaction event sequences during screen reader sessions. Each addresses a narrow slice: structural markup criteria in dynamic content or screen-reader-specific patterns.

The most closely related work targets keyboard accessibility directly. An early approach (Chiou et al., 2021) detected and localized keyboard accessibility failures by crawling pages and verifying that all interactive elements are reachable. BAGEL (Chiou et al., 2023a) extended this with dynamic crawling and visual analysis of the rendered page layout to detect navigation barriers for SC 2.4.3 (Focus Order) and SC 2.4.7 (Focus Visible). BAGEL demonstrates that dynamic browser analysis can detect focus-behavior

violations, but does not extend to measuring focus-indicator size or contrast, nor to detecting whether a focused element is visually obscured by overlapping content properties that require a different probing strategy entirely. A companion study (Chiou et al., 2023b) targeted keyboard accessibility failures inside modal dialogs, including focus-trap violations (states where keyboard focus cannot leave a component) and incorrect focus restoration. All three are limited to one or two criteria and a restricted interaction scope: none evaluates focus-indicator size and contrast (SC 2.4.13) or focus obscuration by overlapping content (SC 2.4.11 and 2.4.12).

One study (López-Gil and Pereira, 2025) showed that LLMs can evaluate several WCAG criteria previously considered too complex for automation, by reasoning over rendered HTML. GenA11y (He et al., 2025) extends this to 37 criteria, sending rendered screenshots and HTML source to an LLM (GPT-4o) (OpenAI, 2023) and prompting it against each criterion. Across these criteria it achieved 94.5% precision and 87.6% recall. However, GenA11y scored 0% recall on SC 2.4.3 and SC 2.4.7, the two focus-behavior criteria it attempted. This is not a model limitation; it is architectural. A screenshot captures a single visual state. Focus indicators and tab-order relationships emerge only during keyboard interaction and are invisible to any snapshot-based approach.

To our knowledge, NavA11y is the first tool to cover all five WCAG 2.4 focus-behavior success criteria. SC 2.4.11, SC 2.4.12, and SC 2.4.13 are unaddressed by any prior tool.

6 Threats to Validity

The primary internal validity threat concerns the composition of D_d , which contains 16 author-constructed pages. These pages were constructed by the same team that developed the detection logic and may therefore be skewed toward violation patterns that the implementation already handles. The six GDS test pages (Government Digital Service, UK Cabinet Office, 2024b) were not visible during tool development. NavA11y achieves 100% precision and recall on all six independently, providing a partial check against this bias. A real production page may trigger unanticipated combinations of CSS overrides, dynamic overlays, and unexpected browser behaviors not present in the test set. Whether this produces false positives or missed detections cannot be determined from D_d alone. The D_r manual audit (Section 4.3) addresses precision on production sites, but recall cannot be measured there without exhaustive ground-truth la-

bellings of all violations on each site.

The 26 websites were taken from the Semrush top-website ranking (Semrush, 2024), a corpus of high-traffic commercial sites. Violation rates on other types of website, such as government portals, small-business pages, and academic institutions, may differ. Reusing the corpus from (He et al., 2025) supports reproducibility but inherits any selection bias toward large commercial sites present in that ranking. This threat is partially mitigated by the fact that detection thresholds are derived from WCAG criterion definitions rather than tuned to patterns observed in D_r .

NavA11y snapshots the set of focusable elements once at page load and does not monitor subsequent DOM mutations. On pages that employ asynchronous content loading or infinite scroll, elements inserted after the initial snapshot are never traversed. Scroll-triggered DOM changes between the pre-focus and post-focus snapshots can corrupt style comparisons, producing incorrect verdicts for all element-level SCs. This limitation affects both accuracy and scalability. The current evaluation partially mitigates this by excluding identified infinite-scroll artifacts from the labeling dataset (Section 4.3). Future work will integrate MutationObserver-based (MDN Web Docs, 2024) monitoring to detect DOM changes between snapshots, adopt incremental analysis that re-snapshots affected elements after mutations settle, and implement synchronization barriers that pause traversal until pending network requests and layout reflows complete.

7 Conclusion

This paper has demonstrated that all five WCAG 2.4 focus-behavior success criteria can be detected automatically through dynamic analysis. The proposed approach analyzes accessibility violations through two phases. The first simulates a full Tab key traversal to capture navigation order for SC 2.4.3. The second directly inspects each focusable element's rendered focus state and obscuration for the remaining four SCs. Each detection threshold is derived from WCAG specification text or validated through empirical pilot studies, ensuring that each verdict is based on defined criteria rather than approximation.

On the 32 controlled test cases, our approach detects every violation and raises no false positives. On 26 production websites, SC 2.4.13 violations appeared on every site; most websites rely on browser-default `:focus` outlines (the built-in styles applied when no custom focus style is declared), which routinely fall below the required indicator thickness and

contrast ratio. SC 2.4.12 violations affected 81% of sites due to persistent overlays covering focused elements. A manual validation by two independent front-end engineers confirms 90% precision on a six-site stratified sample, establishing that the reported violations represent true positive cases for keyboard users. In addition to the approach, this paper releases a labeled dataset (D_d) of 22 test pages covering all five focus-behavior SCs, including pass cases, boundary cases, and obscuration scenarios that are absent from prior public datasets. Future work should extend the analysis to dynamic DOM mutations (content injected by infinite scroll or modal dialogs) and multi-step interactive widgets, and evaluate coverage on authenticated flows.

ACKNOWLEDGEMENTS

This research is supported by the Fellowship from ICT Division, Government of Bangladesh; No-56.00.0000.052.33.002.24-61, dated 13.04.2026.

REFERENCES

- Byrt, T., Bishop, J., and Carlin, J. B. (1993). Bias, prevalence and kappa. *BMC Medical Research Methodology*, 13(5):423–424.
- Chiou, P. T., Alotaibi, A. S., and Halfond, W. G. J. (2021). Detecting and localizing keyboard accessibility failures in web applications. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 618–629. ACM.
- Chiou, P. T., Alotaibi, A. S., and Halfond, W. G. J. (2023a). BAGEL: An approach to automatically detect navigation-based web accessibility barriers for keyboard users. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI)*. ACM.
- Chiou, P. T., Alotaibi, A. S., and Halfond, W. G. J. (2023b). Detecting dialog-related keyboard navigation failures in web applications. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 1362–1373. IEEE.
- Deque Systems (2024). axe-core: accessibility testing engine.
- Durgam, F., Grigera, J., and Garrido, A. (2025). Dynamic detection of accessibility smells. *Univer-*

- sal Access in the Information Society*, 24:281–292.
- Fernandes, N., Costa, D., Neves, S., Duarte, C., and Carriço, L. (2012). Evaluating the accessibility of rich internet applications. In *Proceedings of the 9th International Cross-Disciplinary Conference on Web Accessibility (W4A)*. ACM.
- Fischer, T., Lundell, B., and Gamalielsson, J. (2025). Coverage of web accessibility guidelines provided by automated checking tools. *Universal Access in the Information Society*, 24(4):3615–3637.
- Google LLC (2024). Lighthouse: automated auditing, performance metrics, and best practices for the web.
- Government Digital Service, UK Cabinet Office (2024a). Accessibility monitoring of public sector websites and mobile apps from 2022 to 2024.
- Government Digital Service, UK Cabinet Office (2024b). GDS accessibility tool audit.
- He, Z., Huq, S. F., and Malek, S. (2025). Enhancing web accessibility: Automated detection of issues with generative AI. *Proc. ACM Softw. Eng.*, 2(FSE).
- Internet Live Stats (2024). Total number of websites. Internet Live Stats. Accessed: 2026-03-16.
- Kendall, M. G. (1938). A new measure of rank correlation. *Biometrika*, 30(1–2):81–93.
- Landis, J. R. and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.
- LASIGE, Faculdade de Ciências da Universidade de Lisboa (2024). QualWeb: An automated web accessibility evaluator. <https://qualweb.di.fc.ul.pt>. Accessed: 2026-03-15.
- López-Gil, J.-M. and Pereira, J. (2025). Turning manual web accessibility success criteria into automatic: an LLM-based approach. *Universal Access in the Information Society*, 24(1):837–852.
- MDN Web Docs (2024). MutationObserver. Mozilla Developer Network. Accessed: 2026-03-16.
- MDN Web Docs (2026a). Document: `elementsFromPoint()` method. Mozilla Developer Network. Accessed: 2026-03-08.
- MDN Web Docs (2026b). Window: `getComputedStyle()` method. Mozilla Developer Network. Accessed: 2026-03-08.
- Microsoft (2024). Playwright: Fast and reliable end-to-end testing for modern web apps. <https://playwright.dev>.
- OpenAI (2023). GPT-4 technical report. Technical report, OpenAI. arXiv:2303.08774.
- pally contributors (2024). pally: automated accessibility testing tool.
- Semrush (2024). Semrush – online marketing can be easy. Retrieved August 23, 2024.
- W3C (2018). CSS transitions level 1. W3C Working Draft.
- W3C (2023a). C27: Making the DOM order match the visual order. W3C WCAG Techniques.
- W3C (2023b). F44: Failure of SC 2.4.3 due to using `tabindex` to create a tab order that does not preserve meaning and operability. W3C WCAG Techniques.
- W3C (2023c). F78: Failure of SC 2.4.7 due to styling element outlines and borders in a way that removes or renders non-visible the visual focus indicator. W3C WCAG Techniques.
- W3C (2023d). G1: Adding a link at the top of each page that goes directly to the main content area. W3C WCAG Techniques.
- W3C (2023e). Understanding success criterion 2.4.3: Focus order. W3C Working Group Note.
- W3C (2023f). WCAG 2.2 success criterion 2.4.11: Focus not obscured (minimum). W3C Recommendation.
- W3C (2023g). WCAG 2.2 success criterion 2.4.12: Focus not obscured (enhanced). W3C Recommendation.
- W3C (2023h). WCAG 2.2 success criterion 2.4.13: Focus appearance. W3C Recommendation.
- W3C (2023i). WCAG 2.2 success criterion 2.4.3: Focus order. W3C Recommendation.
- W3C (2023j). WCAG 2.2 success criterion 2.4.7: Focus visible. W3C Recommendation.
- W3C (2023k). Web content accessibility guidelines (WCAG) 2.2. W3C Recommendation.
- WebAIM (2024a). WAVE web accessibility evaluation tool.
- WebAIM (2024b). The WebAIM million: An annual accessibility analysis of the top 1,000,000 home pages. WebAIM. Accessed: 2026-03-14.
- WebAIM (2025). The WebAIM million: The 2025 report on the accessibility of the top 1,000,000 home pages. WebAIM. Accessed: 2026-03-15.
- World Health Organization (2023). Disability and health. WHO Fact Sheet. Accessed: 2026-03-15.